AstraLEO Explore SAMV71 Peripherals, etc.

Build using GCC and C++20. Using FreeRTOS (FreeRTOSv202604.01-LTS).

Current firmware version: **1.4.1** (`IDE_SW_VERSION` in `CMakeLists.txt`).

Building

The build is driven by `CMakePresets.json` (Ninja, output in `build/<preset>/`, executables in `bin/`). Each preset selects four cache variables that pick which source directories are compiled in:

Preset	APP	TARGET	BOARD	PLATFORM
`V71_EK_hello_world`	`Hello_world`	`V71_EK_hello_world`	`V71_EK`	`SamV71` (produces `.elf`)
`Win_hello_world`	`Hello_world`	`Win_hello_world`	`Win11`	`Windows`
`Unit_test`	—	`Unit_test`	`Win11`	`Mockup`
`Win_ethernet_test` / `V71_ethernet_test`	—	(on hold)	`Win11` / `V71_EK`	`Windows` / `SamV71`

```
cmake --preset V71_EK_hello_world
cmake --build build/V71_EK_hello_world
```

SPI (IDE3380 ASIC interface, V1.3)

`SamV71_SPI` (`src/Platform/SamV71_platform/`) talks to up to five IDE3380 ASICs:

- **Bitrate** 2 Mbps (was 100 kbps in V1.2).
- **Mode** CPOL=0 (clock idles low), NCPHA=1 (data captured on the leading edge, changed on the trailing edge).
- **Chip select** is asserted *before* the SPI peripheral is enabled for each transfer.
- **Pins** (`V71_EK_pin_table.cppm`): MISO has a pull-down, MOSI a pull-up.
- **ASIC selection** is 1-based on the console: `AT+ASIC=<1..5>` selects, `AT+ASIC` reports the current selection (both acknowledge with the 1-based number).

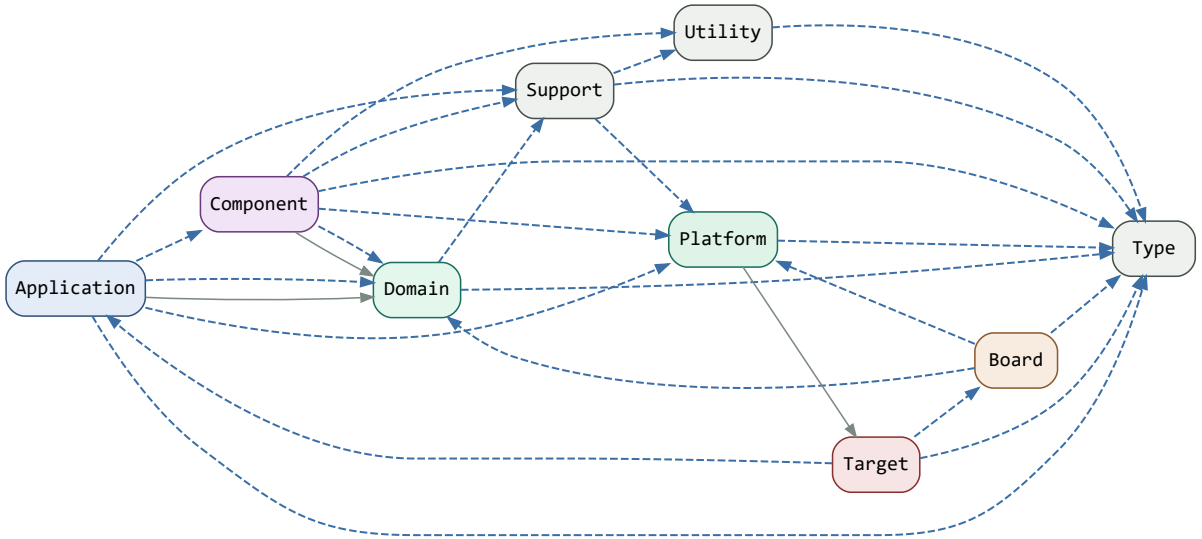
ADC and DAC (SiPM bias, work in progress)

`Bias_DAC` and `Bias_ADC` (`src/Component/SiPM_bias/`) reach the hardware through `Abstract_board::get_DAC()` / `get_ADC()`, implemented on the V71 by `SamV71_DAC` and `SamV71_ADC` (`src/Platform/SamV71_platform/`):

- **`SamV71_DAC`** drives DACC channel 0 and keeps the most recent setting for read-back.
- **`SamV71_ADC`** is still being brought up. It configures AFEC0 for the on-chip temperature sensor channel with a software trigger, and `get()` returns the last value stored by the end-of-conversion callback.
- **Pins** (`V71_EK_pin_table.cppm`): `DAC0` on PB13, `ADC0_0` on PD30.
- **Console**: `AT+CAL_DAC=<value>` sets the raw DAC value and `AT+CAL_DAC` reads it; `AT+CAL_DAC_V35`, `AT+CAL_DAC_V40`, `AT+CAL_ADC_V35` and `AT+CAL_ADC_V40` set the calibration setpoints (see the command table below).

Top-level modules

- **`Application/`** — the app itself: `Hello_world` (composition root, wired up in `main()`, owning `Detector`, `Service_provider`, `SpectraNode_console` and the heartbeat task), `Provider` (the data providers its `Service_provider` routes requests to) and `Support` (cadence control plus C helper code, e.g. the gamma peak detector and isotope table).
- **`Board/`** — board support packages, one per board (`V71_EK` evaluation kit, `Win11` host simulation; `AL_EM` holds only a pin table/config so far), wiring together the platform, components and pin table for that board.
- **`Component/`** — drivers for discrete hardware parts (`IDE3380`, `SiPM_bias`, `STTS22H`), independent of any one board.
- **`Domain/`** — application-domain types and the generated command/keyword dictionary (`Domain/Generated_code`) that ties the console/CLI layer to those types.
- **`Platform/`** — MCU- and RTOS-facing code: `SamV71_platform` (and its `SAMV71Q21B` startup/SDK layer), `Windows_platform` and `Mockup_platform` (host builds), `Common_platform` and `FreeRTOS_platform`. `STM32U575RG` and `WIN32_mockup` are also here, but no preset builds them.
- **`Support/`** — cross-cutting infrastructure: the console support, the heartbeat task (`Hearbeat/`), the CLI parser and instruction dispatch (`Plumbing/`), and the configuration/attribute repository.
- **`Target/`** — top-level build targets that assemble an application, board and platform into one firmware image (`V71_EK_hello_world`, `Win_hello_world`; inactive targets live in `Target/On_hold/`).
- **`Type/`** — shared type definitions: abstract hardware interfaces (`Abstract/`), basic value/status types (`Basic/`) and generic transfer-request types (`Generic/`).
- **`Utility/`** — small, dependency-free helpers (ring buffer, simple string/math) used throughout the rest of the tree.



Top-level module dependency graph -- one node per top-level src/ directory

reset V71_EK_hello_world: APP=Hello_world, TARGET=V71_EK_hello_world, BOARD=V71_EK, PLATFORM=SamV71) (arrow: directory -> directory it depends c

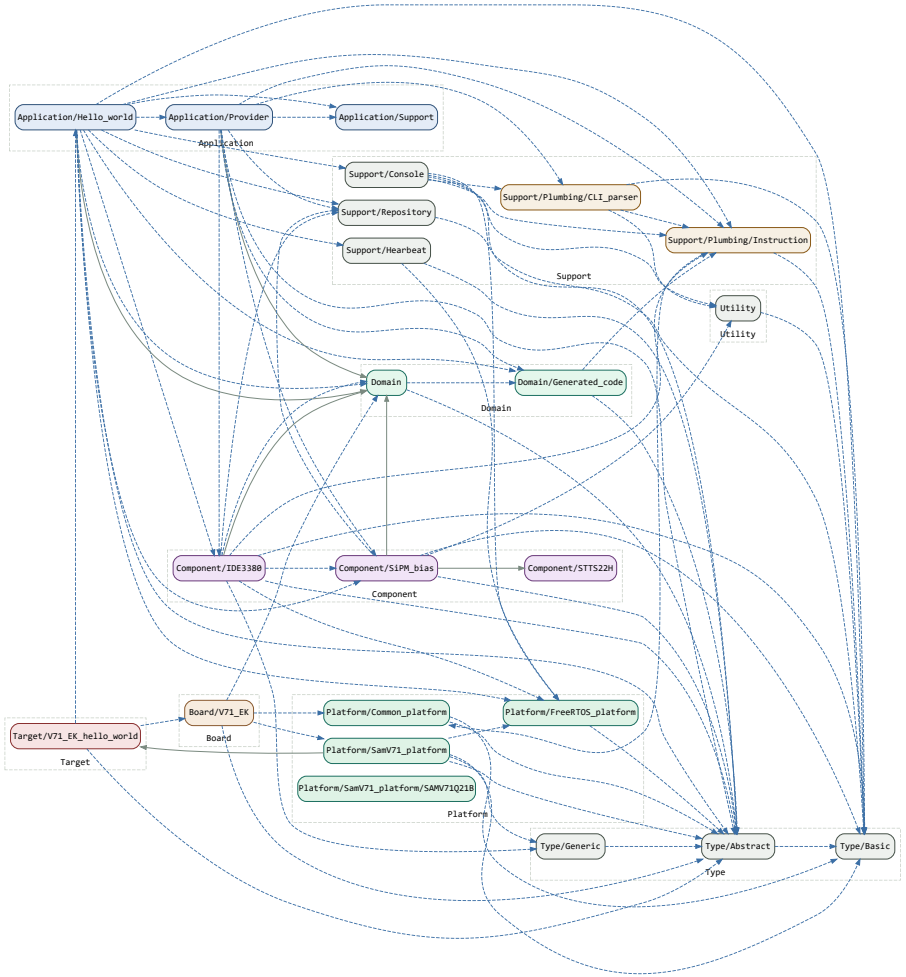
solid = #include dependency dashed = C++20 module import bold red = two-way (circular) dependency between the pair

Rolled up from doc/module_dependencies.svg (per-source-directory graph).

The graph above collapses every source directory into its top-level `src/` area (`Application/`, `Board/`, `Component/`, `Domain/`, `Platform/`, `Support/`, `Target/`, `Type/`, `Utility/`). The full, per-source-directory graph it's rolled up from:

Dependencies

Covers both `#include` edges (solid) and C++20 module `import` edges (dashed), for the directories CMake actually builds with the `V71_EK_hello_world` preset. Almost every module is now a named C++20 module, so `import` dominates: the only remaining `#include` edges are the plain `Domain/Persistent_parameter_id.h` header (from `Application/Hello_world`, `Application/Provider`, `Component/IDE3380`, `Component/SiPM_bias`), `Component/SiPM_bias` → `Component/STTS22H` (`Sensor_STTS22H.h`) and `Platform/SamV71_platform` → `Target/V71_EK_hello_world` (`FreeRTOSConfig.h`). A module pair with edges in both directions would be drawn bold red; there are currently none.

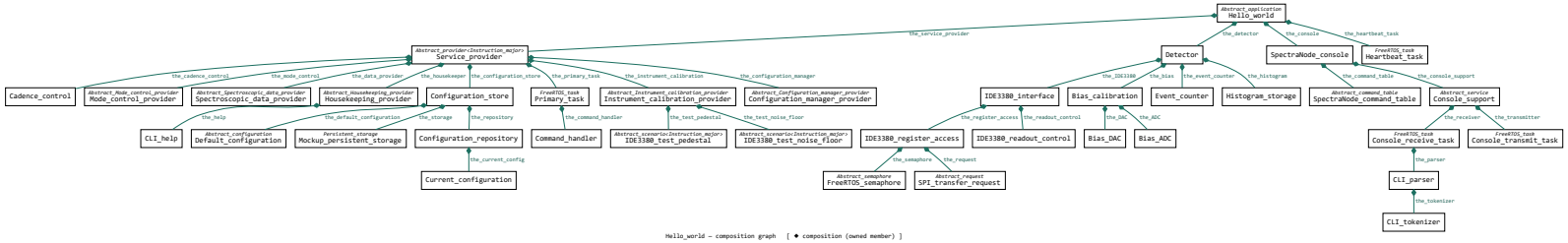


Module dependency graph — one node per source directory actually built by CMake
(preset V71_EK_hello_world: APP=Hello_world, TARGET=V71_EK_hello_world, BOARD=V71_EK, PLATFORM=SamV71) (arrow: module → module it depends on)
solid = #include dependency dashed = C++20 module import bold red = two-way (circular) dependency between the pair

Wiring

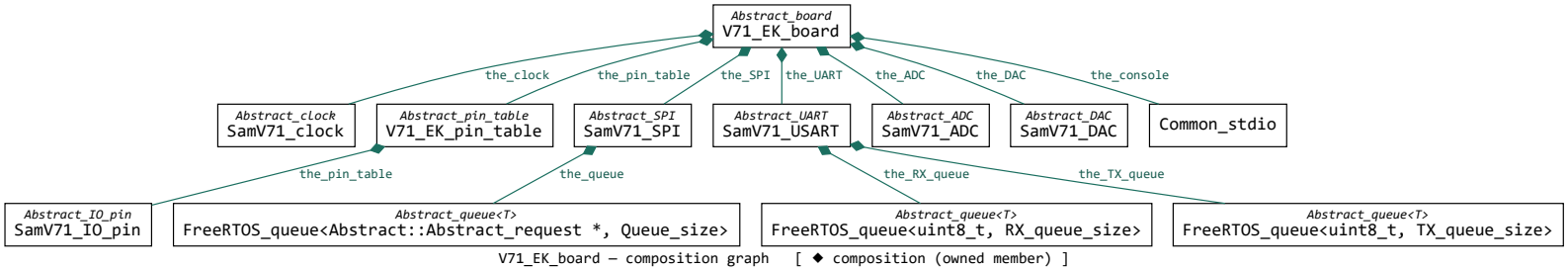
The two concrete objects wired together in `main()` (`V71_EK_hello_world.cpp`), first as composition-only overviews showing just what each one owns, then as the detailed aggregation graphs that add the non-owning references. `Hello_world` is too large for one readable aggregation graph, so it has one each for `Service_provider` and `SpectraNode_console`, two of the four objects it owns.

What `Hello_world` (the application) owns:

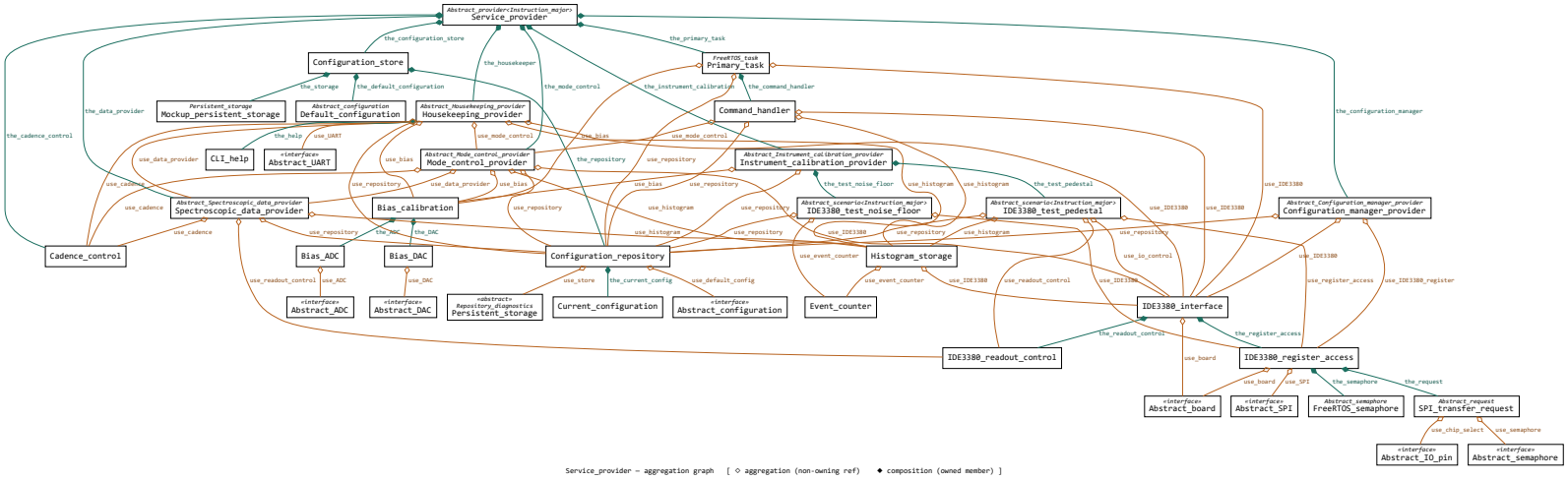


Hello_world — composition graph [◆ composition (owned member)]

What `V71_EK_board` (the board) owns:

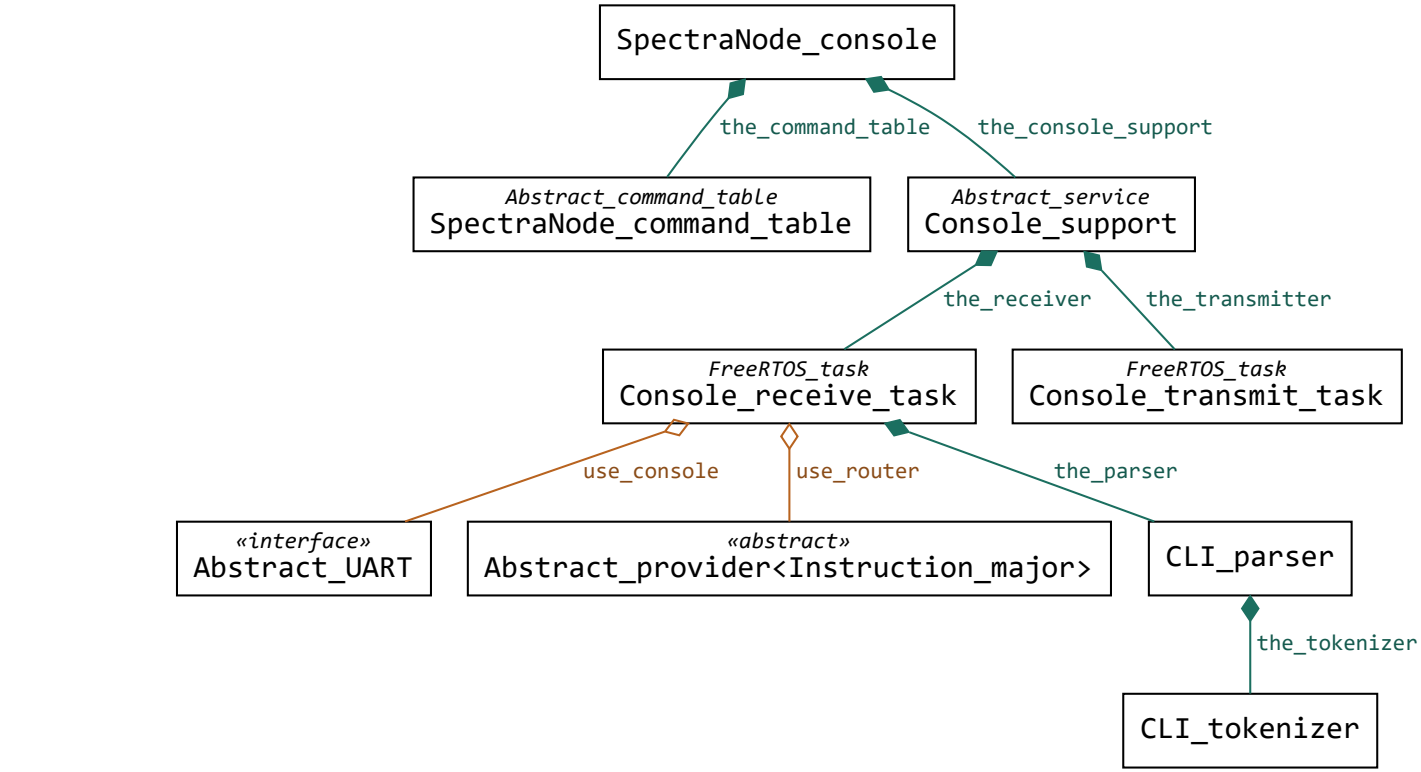


What `Service_provider` (request routing and the data providers) owns and references:



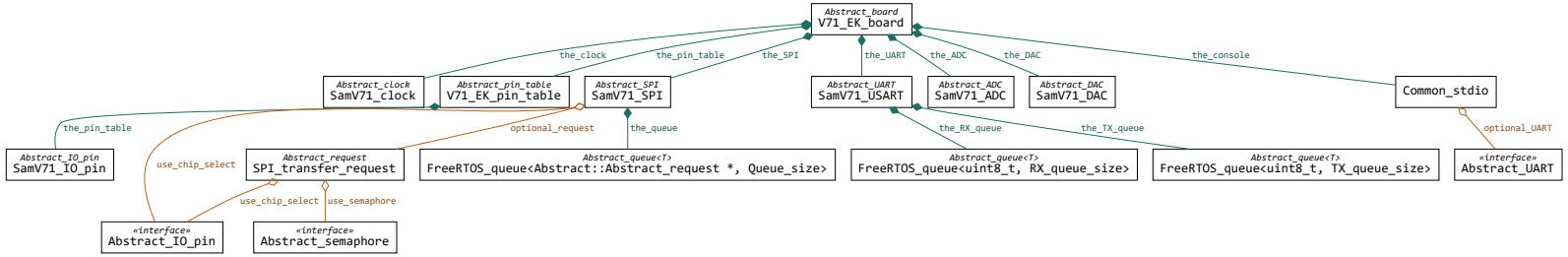
Service_provider — aggregation graph [◇ aggregation (non-owning ref) ◆ composition (owned member)]

What `SpectraNode_console` (the console and its command table) owns and references:



SpectraNode_console — aggregation graph [◇ aggregation (non-owning ref) ◆ composition (owned member)]

What `V71_EK_board` (the board) owns and references:



V71_EK_board — aggregation graph [◇ aggregation (non-owning ref) ◆ composition (owned member)]

SpectraNode command & keyword dictionary

`src/Domain/Generated_code/` is generated from `src/Domain/Definition_SpectraNode/*.xml` via `src/Domain/Definition_SpectraNode/SpectraNode_generator.bat` (XSLT stylesheets in `src/Domain/Dictionary_scripts/`), which also re-splices the tables below (via `Update_readme_dictionary.ps1`, next to the generator) from `SpectraNode_command.md`, `SpectraNode_keyword.md` and `SpectraNode_structure.md` every time it runs — the content between each `<!-- BEGIN:... -->` `<!-- END:... -->` marker below is generated, not hand-maintained; edit the source XML and re-run the generator, never this section directly. Shown open by default (collapse with the triangle if you don't need them).

▼ **Commands** (FW1038 SpectraNode command version 3)

Provider: Mode_control

Command	Description
`AT+DEMO_CADENCE=<seconds(int 0..1000000)>`	Set telemetry interval (s) in Demonstration Mode.
`AT+DEMO_CHANNEL=<channel(int 0..18)>`	Set channel to be used in live view demo. 0=inhibit readout, 1..16=input channel, 17=analog summing, 18=digital summing.
`AT+IDLE_CADENCE=<seconds(int 0..1000000)>`	Set telemetry interval (s) in Idle Mode.
`AT+MODE_DEMO`	Set demonstration mode.
`AT+MODE_IDLE`	Set idle mode.
`AT+MODE_NOMINAL`	Set nominal mode.
`AT+NOMINAL_ALARM=<micro_sievert(int 0..1000000)>`	Set alarm threshold (uSv/h).
`AT+NOMINAL_CADENCE=<seconds(int 0..1000000)>`	Set telemetry interval (s) in Nominal Mode.

Provider: Spectroscopic_data

Command	Description
`AT+FORMAT_CPS`	Send event counters.
`AT+FORMAT_N42`	Use complex histogram format (N42).
`AT+FORMAT_R6`	Use simple histogram format (R6).
`AT+GET_RESET`	Send the histogram and clear the buffers.
`AT+GET`	Send the histogram.
`AT+TIME=<date(int 0..0x7FFFFFFF)>,<time(int 0..1000000)>`	Set date and time.

Provider: Instrument_calibration

Command	Description
`AT+CAL_ADC_V35=<cal_35V(int -100000..-1)>`	Set bias ADC calibration 35V setpoint.
`AT+CAL_ADC_V40=<cal_40V(int -100000..-1)>`	Set bias ADC calibration 40V setpoint.
`AT+CAL_DAC=<data16(int 0..0xFFFF)>`	Set raw DAC value.
`AT+CAL_DAC_V35=<cal_35V(int -100000..-1)>`	Set bias DAC calibration 35V setpoint.
`AT+CAL_DAC_V40=<cal_40V(int -100000..-1)>`	Set bias DAC calibration 40V setpoint.
`AT+CAL_DAC`	Get the raw DAC setting.
`AT+CAL_DIAG`	Misc. diagnostic information wrt. calibration.
`AT+CAL_TEST_BIAS`	Find common bias voltage using dark count rate.
`AT+CAL_TEST_NOISE`	Find channel noise floor using threshold scan.
`AT+CAL_TEST_OFFSET`	Find channel offset voltage using fixed source.
`AT+CAL_TEST_PEDESTAL`	Find channel pedestal using forced readout.
`AT+CAL_TRACE`	Toggle diagnostic trace on/off.

Provider: Configuration_manager

Command	Description
`AT+ASIC=<asic_nbr(int 1..5)>`	Select ASIC 1-5.
`AT+ASIC_DUMP`	Dump all the ASIC registers.
`AT+ASIC_LOAD`	Update all ASIC registers from current configuration.
`AT+ASIC_REG=<reg_addr(int 0..32)>,<data32(int 0..0x7FFFFFFF)>`	Write the ASIC register.
`AT+ASIC_REG=<reg_addr(int 0..32)>`	Read the ASIC register.
`AT+ASIC`	Show current ASIC selection.
`AT+CONFIG_APPLY`	Apply the current configuration.
`AT+CONFIG_ASIC=<asic_nbr(int 1..5)>,<reg_addr(int 0..32)>,<data32(int 0..0x7FFFFFFF)>`	Set the current configuration register value.
`AT+CONFIG_ASIC=<asic_nbr(int 1..5)>,<reg_addr(int 0..32)>`	Get the current configuration register value.
`AT+CONFIG_CLEAN`	Clean the persistent storage and set the default configuration.
`AT+CONFIG_DUMP`	Dump the current configuration.
`AT+CONFIG_LOAD`	Load the current configuration from the persistent storage.
`AT+CONFIG=<offset(int 0..4095)>,<data32(int 0..0x7FFFFFFF)>`	Set configuration attribute value.
`AT+CONFIG=<offset(int 0..4095)>`	Get configuration attribute value.
`AT+CONFIG_SAVE`	Save the current configuration to the persistent storage.

Provider: Housekeeping

Command	Description
`AT+DIAG`	Send miscellaneous diagnostic information.
`AT+ECHO`	Toggle console echo on/off.
`AT+HELP`	Show commands.
`AT+STATUS`	Send mode and other system data.
`AT+TEST`	Initiate a self-test sequence.
`AT+TRACE`	Toggle trace on/off.
`AT+VERSION`	Send version and other build information.

▼ **Keywords** (SpectraNode version 1)

Id	Name	Description
0	`"No key"`	Special purpose
1	`a`	
2	`active`	
3	`adc`	
4	`ADC`	
5	`ALARM`	
6	`APPLY`	
7	`ASIC`	
8	`asic_nbr`	
9	`b`	
10	`bias`	
11	`BIAS`	
12	`CADENCE`	
13	`CAL`	
14	`cal_35V`	
15	`cal_40V`	
16	`calibration`	
17	`channel`	

Id	Name	Description
18	`CHANNEL`	
19	`CLEAN`	
20	`CONFIG`	
21	`Configuration_manager`	
22	`CPS`	
23	`CSV`	
24	`dac`	
25	`DAC`	
26	`data16`	
27	`data32`	
28	`data8`	
29	`date`	
30	`demo`	
31	`DEMO`	
32	`device`	
33	`DIAG`	
34	`DUMP`	
35	`ECHO`	
36	`format`	
37	`FORMAT`	
38	`gain`	
39	`GET`	
40	`HELP`	
41	`Housekeeping`	
42	`idle`	
43	`IDLE`	
44	`INPUT`	
45	`Instrument_calibration`	
46	`integration_time`	
47	`LOAD`	
48	`micro_sievert`	
49	`millis`	
50	`MODE`	
51	`Mode_control`	
52	`N42`	
53	`NOISE`	
54	`nominal`	
55	`NOMINAL`	
56	`offset`	
57	`OFFSET`	
58	`parameter`	
59	`pedestal`	
60	`PEDESTAL`	
61	`R6`	
62	`readout_count`	
63	`REG`	
64	`reg_addr`	
65	`RESET`	
66	`SAVE`	
67	`seconds`	

Id	Name	Description
68	`serial_number`	
69	`simple_R6`	
70	`Spectroscopic_data`	
71	`start_threshold`	
72	`STATUS`	
73	`stop_count`	
74	`TEST`	
75	`time`	
76	`TIME`	
77	`TRACE`	
78	`V35`	
79	`V40`	
80	`VBIAS`	
81	`VERSION`	
255	`"Wildcard"`	Special purpose, used in search

▼ **Structure** (SpectraNode_structure_definition)

Identifier	Type	Default	Description
device.serial_number	data32	0	Device serial number.
device.reg_addr.[0-29]	data32	0x0200'0323	IDE3380 ASIC register default (channel 1-16 control register template; see IDE3380_register_decoder for the rest).
MODE.active	idle|nominal|demo	Key_demo	Active operating mode at startup.
CADENCE.DEMO	seconds	2	Telemetry cadence in Demo mode.
CADENCE.NOMINAL	seconds	2	Telemetry cadence in Nominal mode.
CADENCE.IDLE	seconds	2	Telemetry cadence in Idle mode.
CHANNEL.DEMO	channel	18	Active channel in Demo mode, 0=inhibit, 1-16=input, 17=analog summing, 18=digital summing.
CHANNEL.NOMINAL	channel	18	Active channel in Nominal mode.
CHANNEL.IDLE	channel	18	Active channel in Idle mode.
format.DEMO	no_data|legacy_live_view|simple_R6|only_CPS|complex_N42|housekeeping	Key_simple_R6	Data format in Demo mode.
format.IDLE	no_data|legacy_live_view|simple_R6|only_CPS|complex_N42|housekeeping	Key_simple_R6	Data format in Idle mode.
format.NOMINAL	no_data|legacy_live_view|simple_R6|only_CPS|complex_N42|housekeeping	Key_simple_R6	Data format in Nominal mode.
calibration.parameter.a	data32	100000	Calibration parameter A.
calibration.parameter.b	data32	10000	Calibration parameter B.
calibration.BIAS	data32	0	Bias voltage at 25 degC reference.
calibration.adc.V35	mV	-35000	ADC calibration setpoint at -35V.
calibration.adc.V40	mV	-40000	ADC calibration setpoint at -40V.

Identifier	Type	Default	Description
calibration.dac.V35	mV	-35000	DAC calibration setpoint at -35V.
calibration.dac.V40	mV	-40000	DAC calibration setpoint at -40V.
calibration.integration_time	number	100	Calibration pulse integration time.
calibration.start_threshold	number	100	Calibration start threshold.
calibration.stop_count	number	100	Calibration stop count.
calibration.readout_count	number	100000	Calibration readout count.
calibration.pedestal.[1-16]	data32	50	Per-channel pedestal offset.
calibration.gain.[1-16]	data32	1	Per-channel gain factor.

C++20 modules

Several parts of `src/` (`Type/`, `Support/`, `Application/`, `Component/`, `Board/V71_EK`, the built `Platform/` directories, `Utility/` and `Domain/Generated_code`) are built as named C++20 modules rather than traditional header/source pairs. A quick primer:

- **Module interface unit** — declares and exports the module's public API. File extension is `.cppm`:

```
export module Support.Console_support;   // module name (dots are just part of the name)

import Type.Abstract_service;            // pull in another module

export class Console_support : public Abstract_service { // exported: visible to importers
    ...
};

void log_internal(...);                  // not exported: module-private
```

- **Module implementation unit** — provides definitions for a module declared elsewhere, without re-exporting its interface: `module Support.Console_support;` (no `export`) at the top of a `.cpp` file. Counts as depending on whatever module it implements.
- **Importing a module** — `import Some.Module.Name;` (or `export import ...;` to re-export it through the importing module) instead of `#include`-ing a header. No preprocessor, no include guards, no macro leakage between translation units.
- **Partitions** — a module can be split across files internally with `export module Foo:Partition_name;` / `import :Partition_name;`; partitions are only visible to other pieces of the same module, never to importers of `Foo`.
- **Naming convention used in this repo** — module names mirror their source directory, dotted (e.g. `Support/Console/Console_support.cppm` exports `Support.Console_support`), which is why the dependency graphs above draw `import` edges (dashed) between the same directory nodes as `#include` edges (solid).

This project still mixes plain headers (mainly SDK/RTOS/third-party code, plus a few C-API sources) with modules, hence the graphs above track both `#include` and `import` edges rather than assuming one or the other.